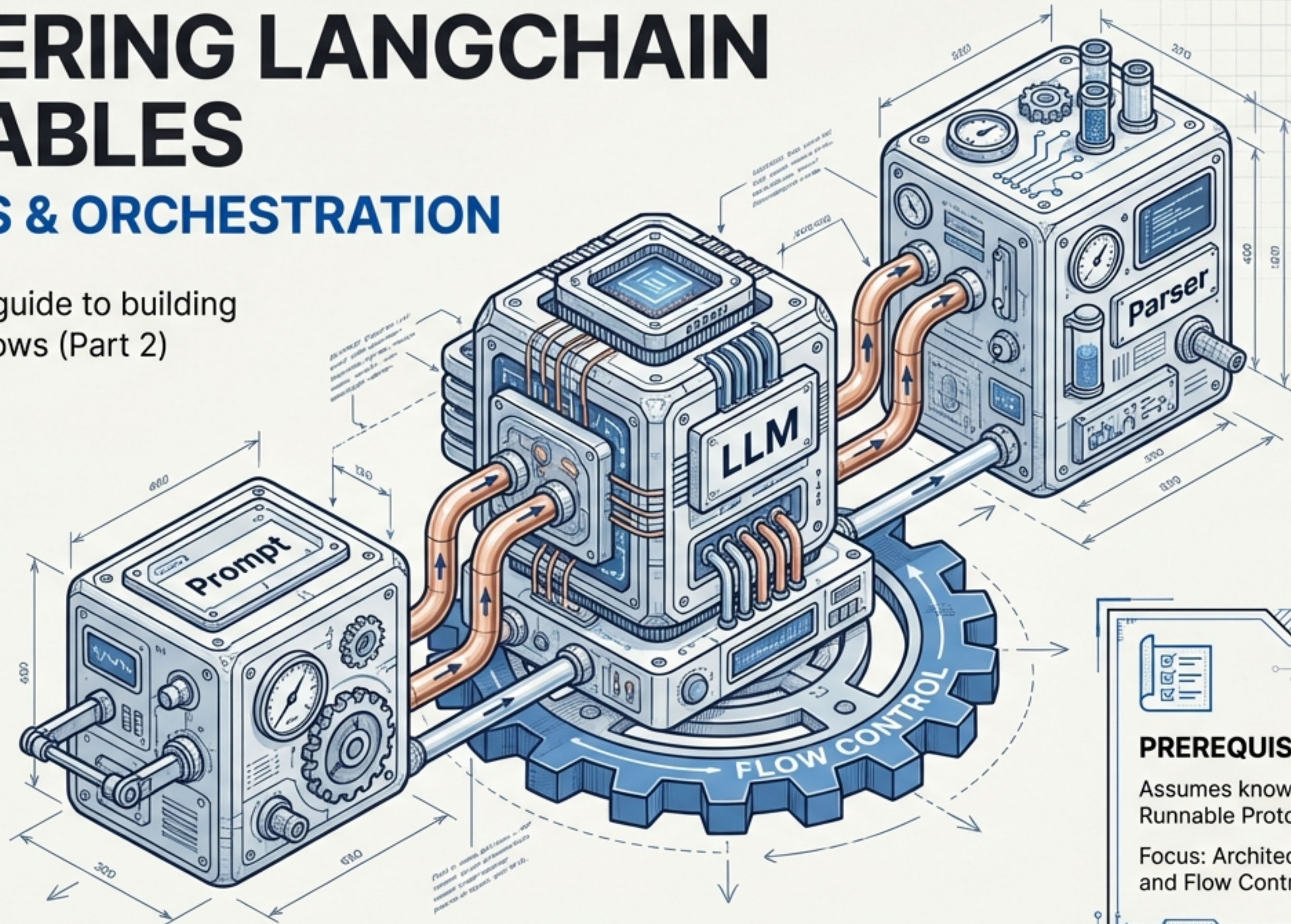


MASTERING LANGCHAIN RUNNABLES

PRIMITIVES & ORCHESTRATION

A comprehensive guide to building
complex AI workflows (Part 2)



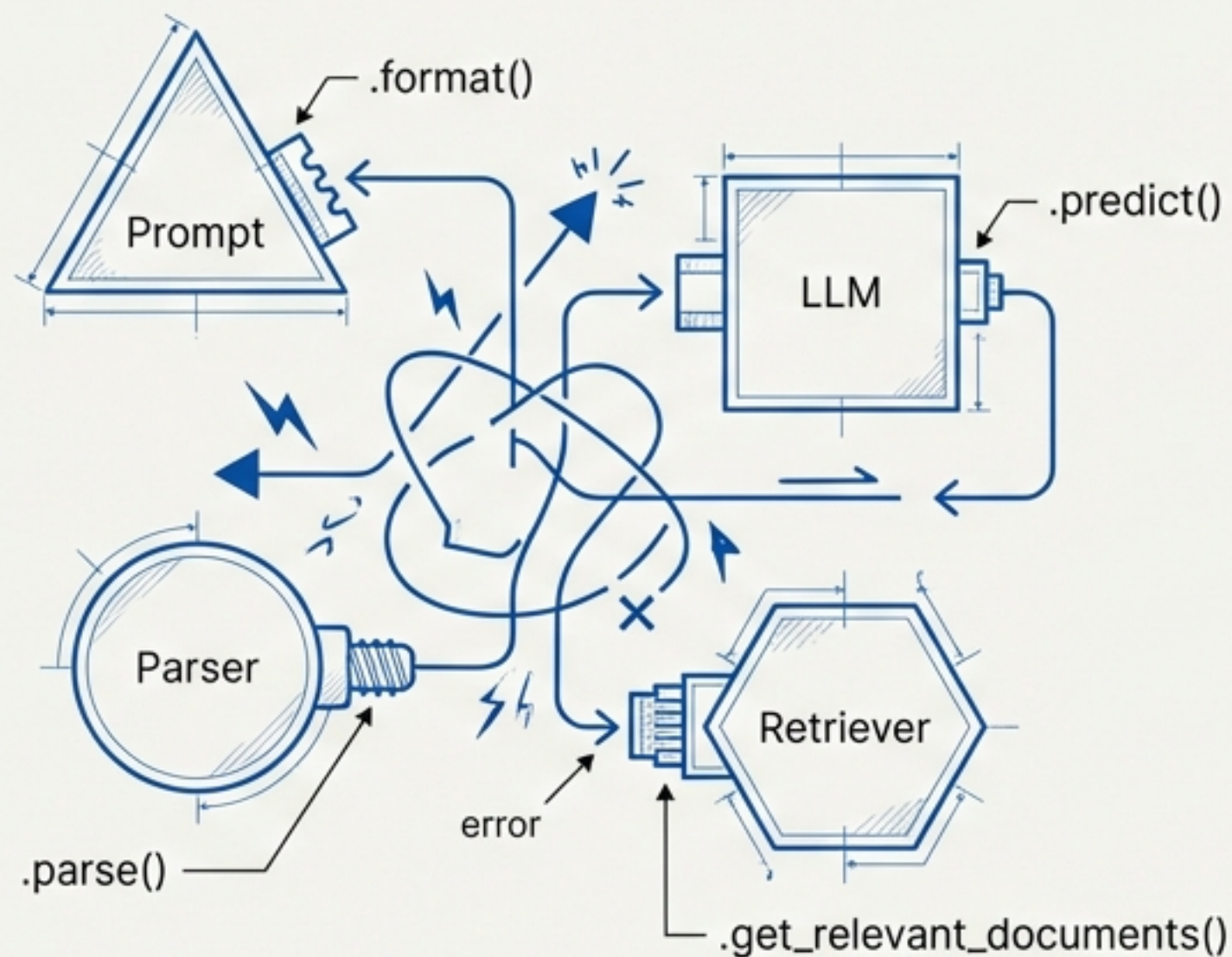
PREREQUISITES:

Assumes knowledge of The
Runnable Protocol (Part 1).

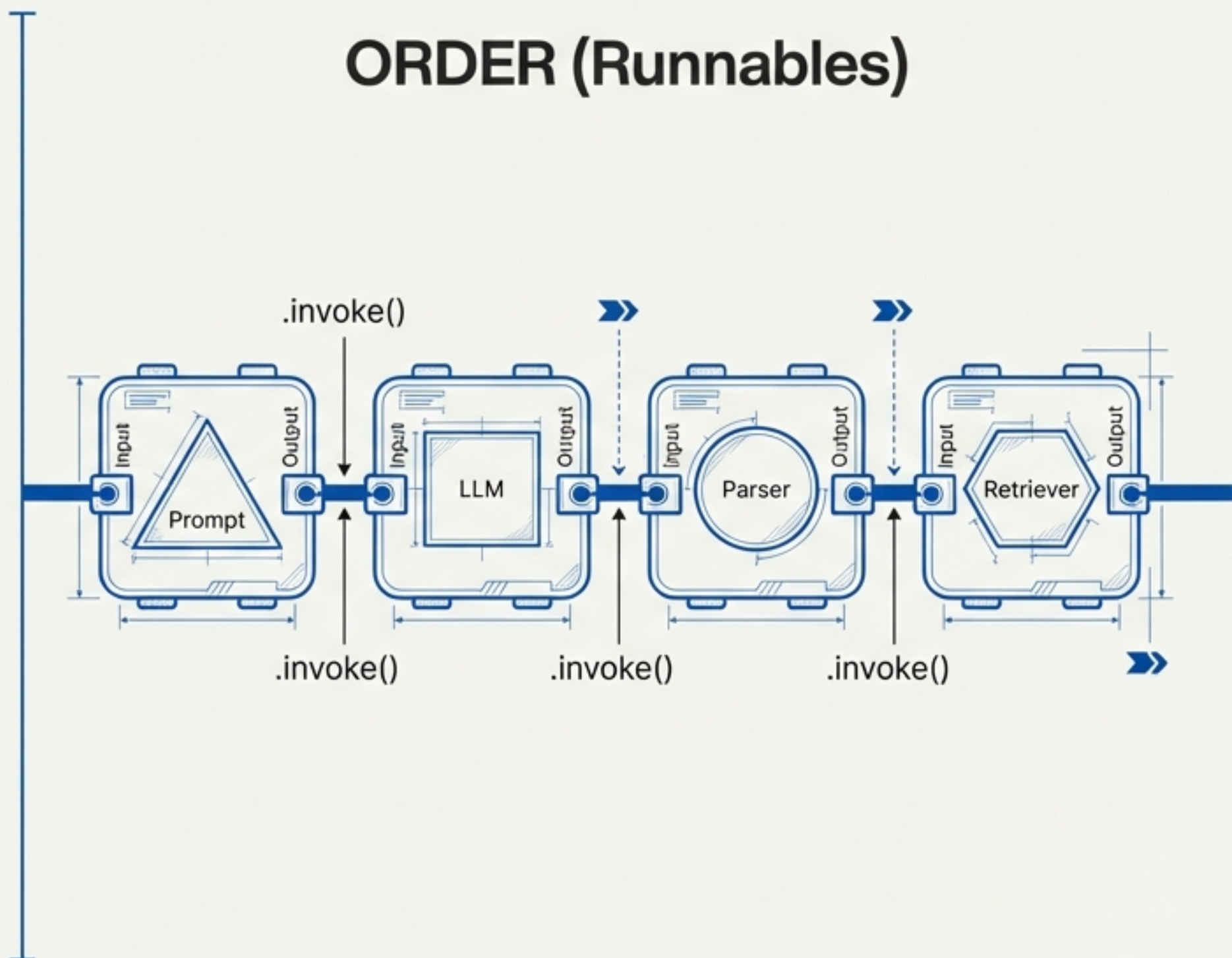
Focus: Architecture, Wiring,
and Flow Control.

THE FOUNDATION: THE POWER OF STANDARDISATION

CHAOS (Legacy)



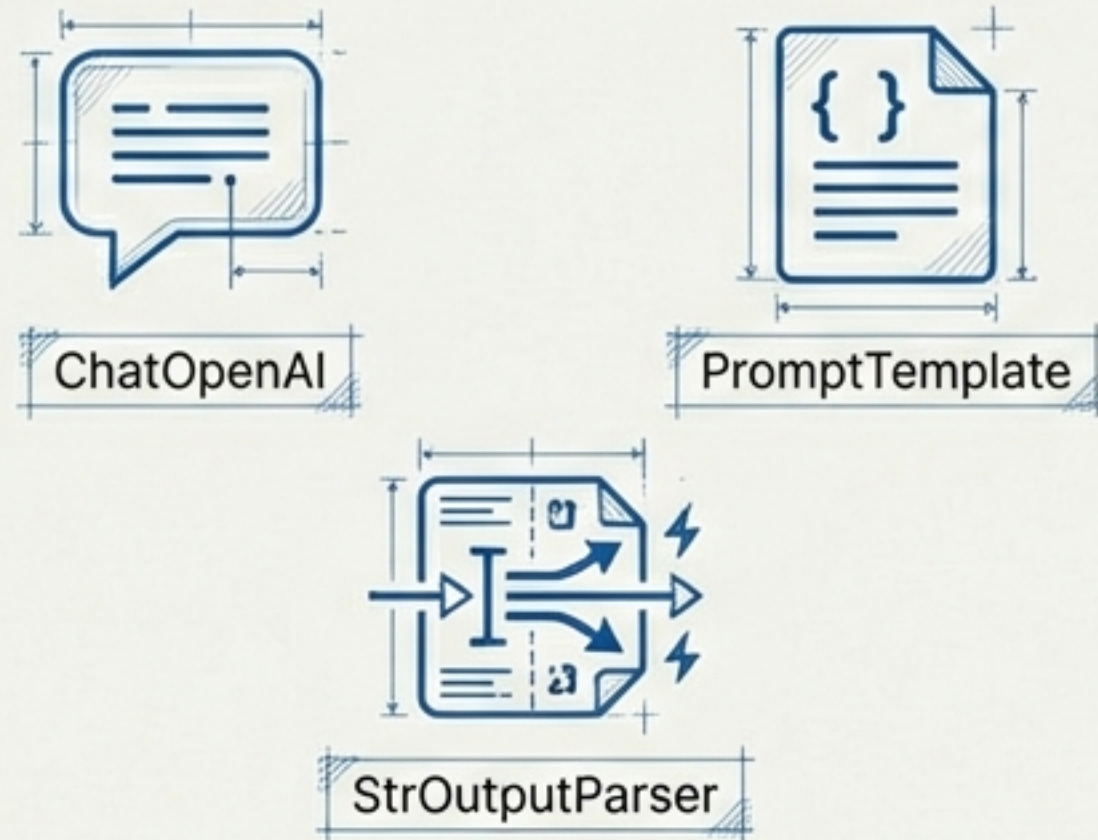
ORDER (Runnables)



THE RUNNABLE ECOSYSTEM

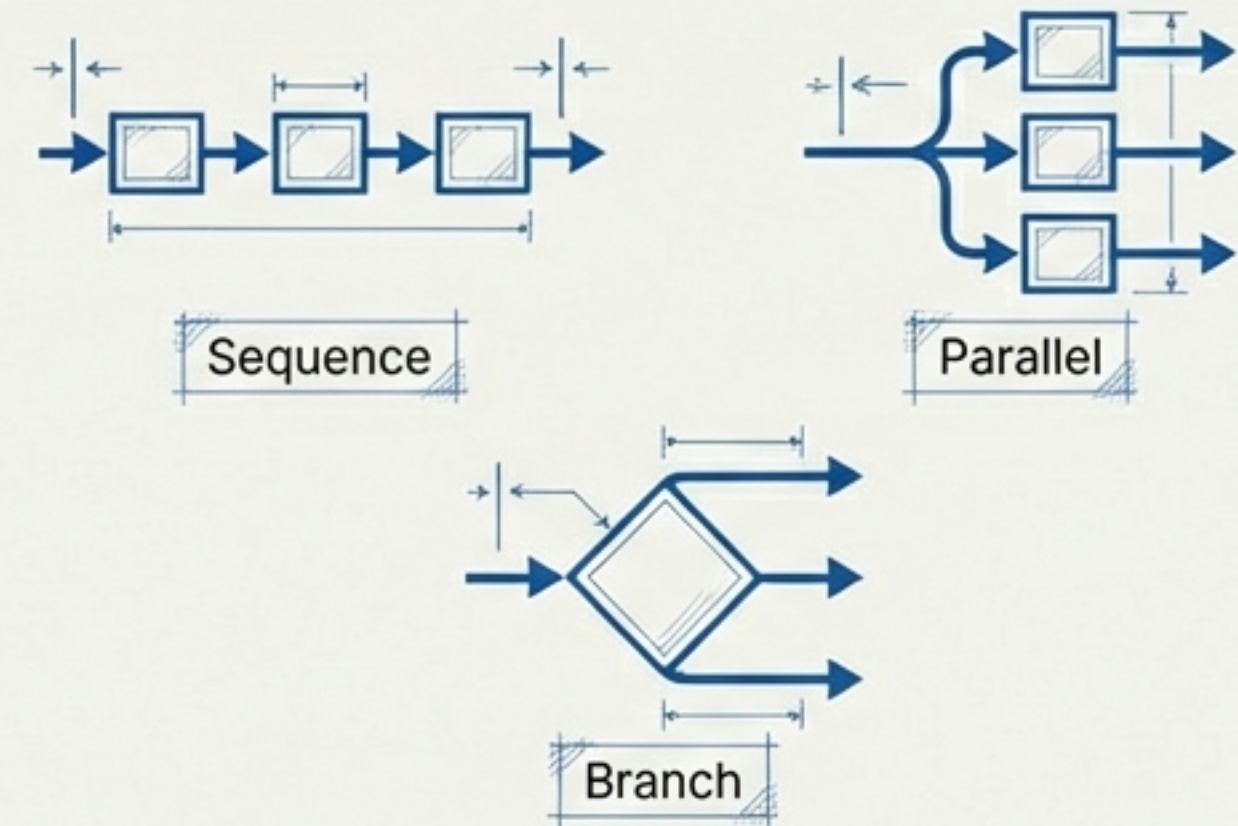
Two distinct categories of building blocks.

1. TASK-SPECIFIC RUNNABLES (The Workers)



Core components converted to Runnables. They perform specific jobs like formatting text or querying models.

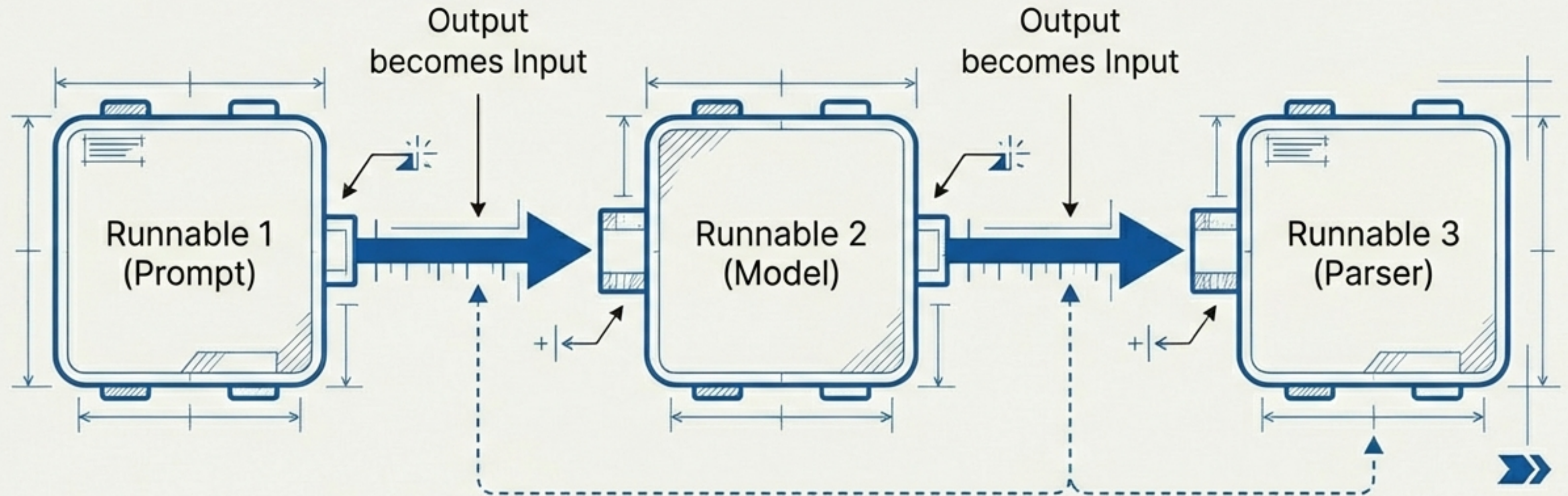
2. RUNNABLE PRIMITIVES (The Managers)



Fundamental blocks that orchestrate flow. They don't generate content; they decide how content moves.

PRIMITIVE 1: RUNNABLE SEQUENCE

The Spine of the Chain



DEFINITION: Connects two or more Runnables sequentially. The output of the previous step becomes the input of the next step automatically.

CODE IN ACTION: THE LINEAR CHAIN

1. Define Workers

```
prompt = PromptTemplate.from_template(  
    "Write a joke about {topic}")
```

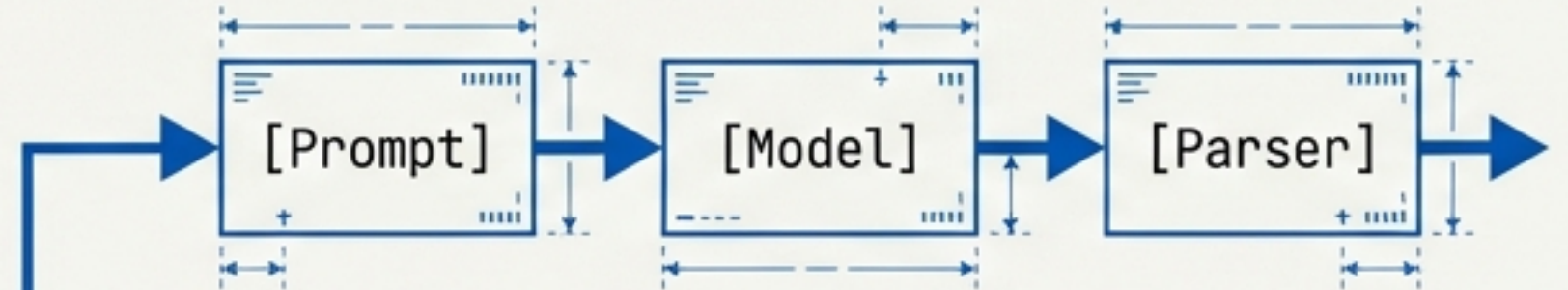
```
model = ChatOpenAI()  
parser = StrOutputParser()
```

2. Define Manager (Sequence)

```
chain = RunnableSequence(  
    first=prompt,  
    middle=[model],  
    last=parser  
)
```

3. Execute

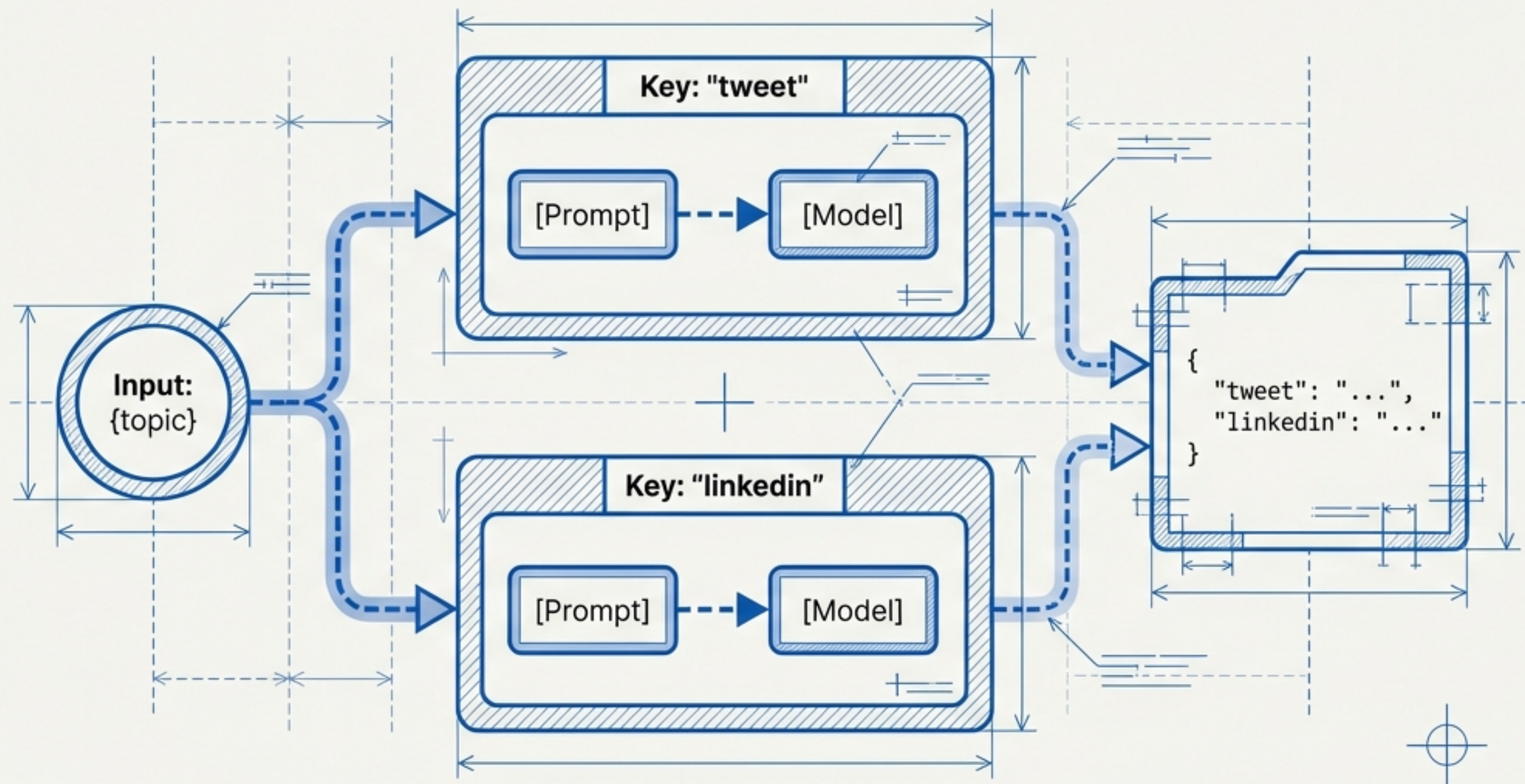
```
chain.invoke({"topic": "AI"})
```



```
> "Why did the AI go to school? To  
improve its neural network!"
```

PRIMITIVE 2: RUNNABLE PARALLEL

Multitasking & Simultaneous Execution



- **Shared Input:** All branches receive the exact same input.
- **Independent Execution:** Branches process simultaneously.
- **Dictionary Output:** Results mapped to keys.

CODE IN ACTION: PARALLEL CONTENT CREATION

```
# Define separate chains
tweet_chain = RunnableSequence(...)
linkedin_chain = RunnableSequence(...)

# Orchestrate in Parallel
parallel_chain = RunnableParallel({
    "tweet": tweet_chain,
    "linkedin": linkedin_chain
})

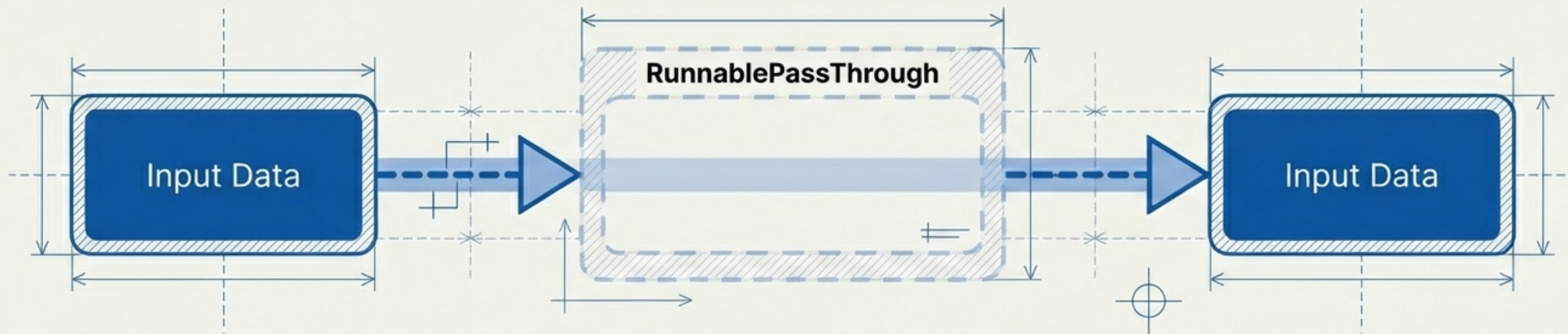
# Input "AI" is broadcast to both
parallel_chain.invoke({"topic": "AI"})
```

Broadcast
Input

```
{
  "tweet": "AI is transforming
the future! #Tech #Future",
  "linkedin": "Artificial
Intelligence is not just a
buzzword. It is a fundamental
shift in how we approach
problem-solving in the modern
enterprise..."
}
```

PRIMITIVE 3: RUNNABLE PASS THROUGH

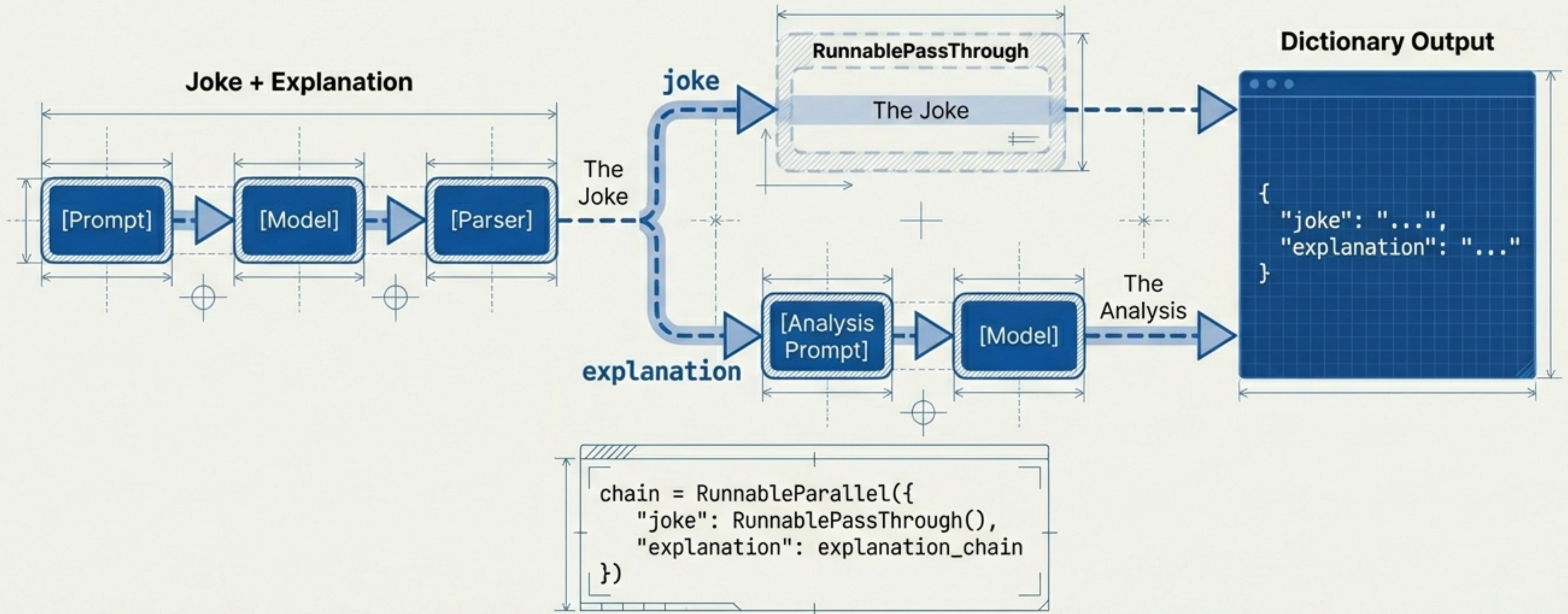
The Transparent Layer



- **THE PROBLEM:** In linear chains, step 3 only sees step 2's output. The original input is lost.
- **THE SOLUTION:** PassThrough acts as a placeholder that forwards data unchanged, allowing you to carry context to later steps.

ADVANCED WORKFLOW: PRESERVING CONTEXT

Combining Sequence, Parallel, and PassThrough



PRIMITIVE 4: RUNNABLE LAMBDA

Integrating Custom Logic



❶ **DEFINITION:** A wrapper that turns ANY standard Python function into a Runnable component.

❷ **USE CASES:**

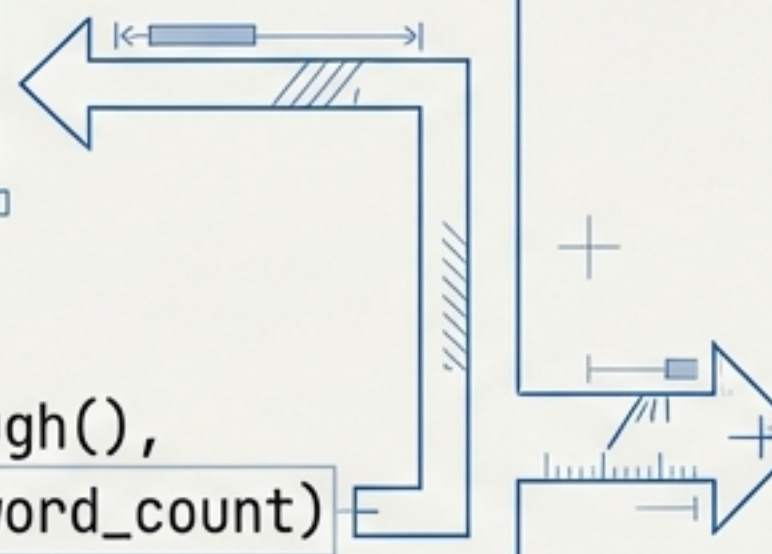
- Pre-processing: Cleaning text, removing HTML tags.
- Post-processing: Calculating word counts, formatting dates.

CODE IN ACTION: MIXING AI AND CODE

Adding a deterministic word counter to a generative chain

PYTHON CODE FRAME

```
def word_count(text):  
    return len(text.split())  
  
chain = RunnableParallel({  
    "joke": RunnablePassThrough(),  
    "count": RunnableLambda(word_count)  
})  
  
chain.invoke("The generated joke text...")
```



JSON OUTPUT FRAME

```
{  
  "joke": "Why did the computer  
go to the doctor? Because it  
had a virus!",  
  "count": 12  
}
```

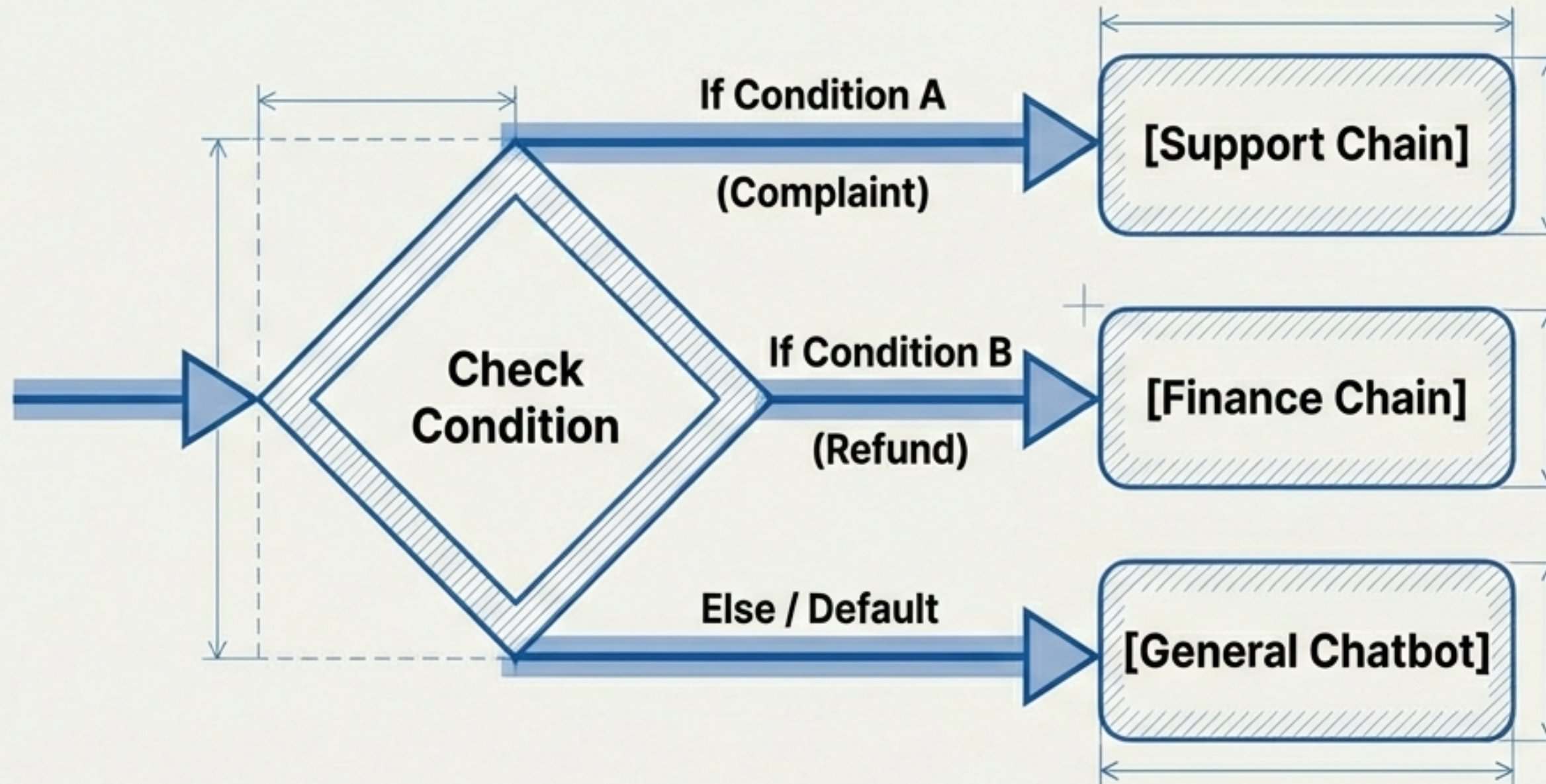
RunnableLambda Output

RunnableLambda Output

Output showing deterministic 'count' from custom logic alongside generative 'joke'

PRIMITIVE 5: RUNNABLE BRANCH

The Decision Maker (Routing Logic)



STRUCTURE:

1. List of (Condition, Runnable) tuples.
2. Final Default Runnable for fallback.

CODE IN ACTION: DYNAMIC ROUTING

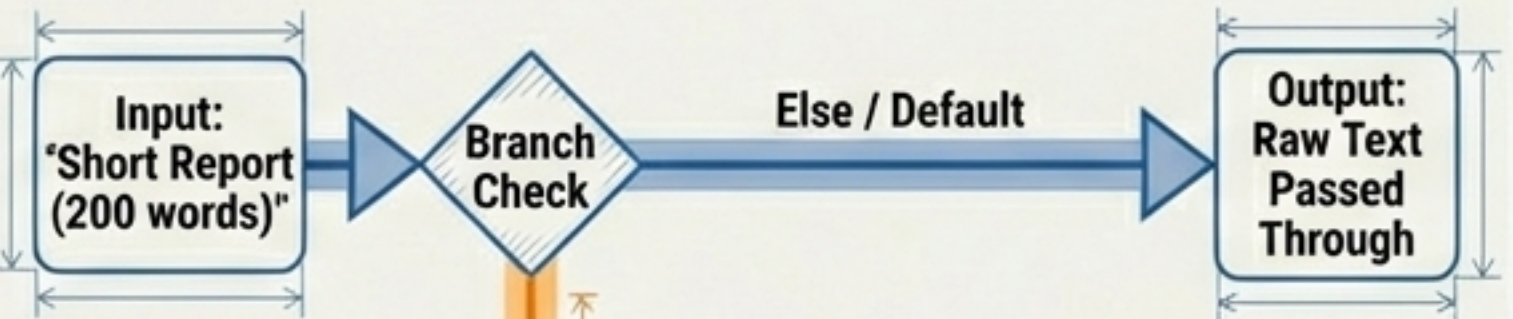
Routing based on input length

PYTHON CODE FRAME

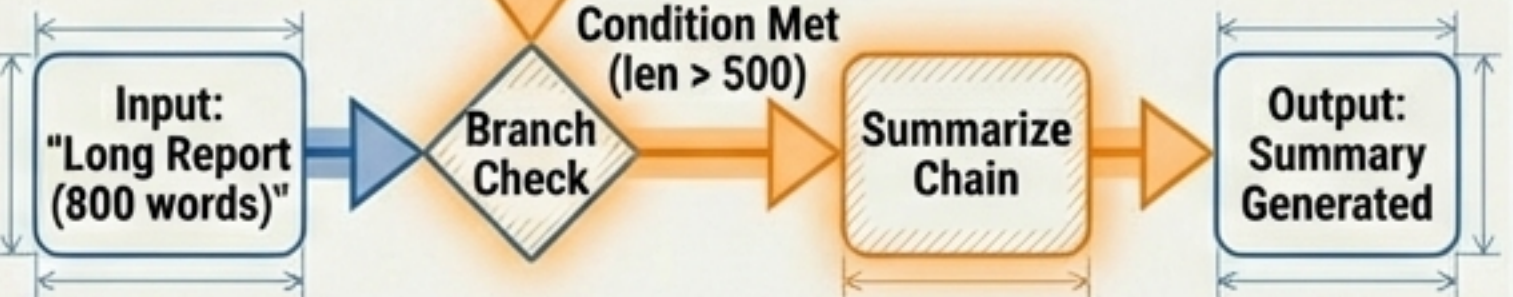
```
branch = RunnableBranch(  
    (lambda x: len(x.split())  
  
    (lambda x: len(x.split()) > 500,  
    summarize_chain),  
    RunnablePassThrough() # Default (Else)  
)
```

SIMULATION FRAME

Strip 1: Short Report



Strip 2: Long Report

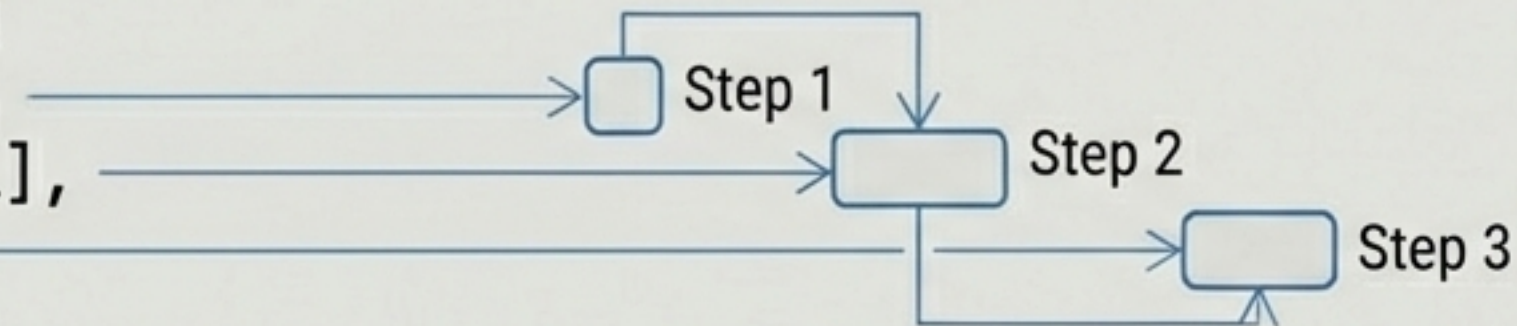


LCEL: THE SYNTAX REVOLUTION

LangChain Expression Language

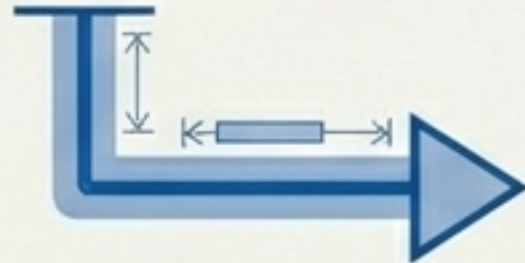
THE OLD WAY (Verbose)

```
RunnableSequence(  
    first=prompt,  
    middle=[model],  
    last=parser  
)
```



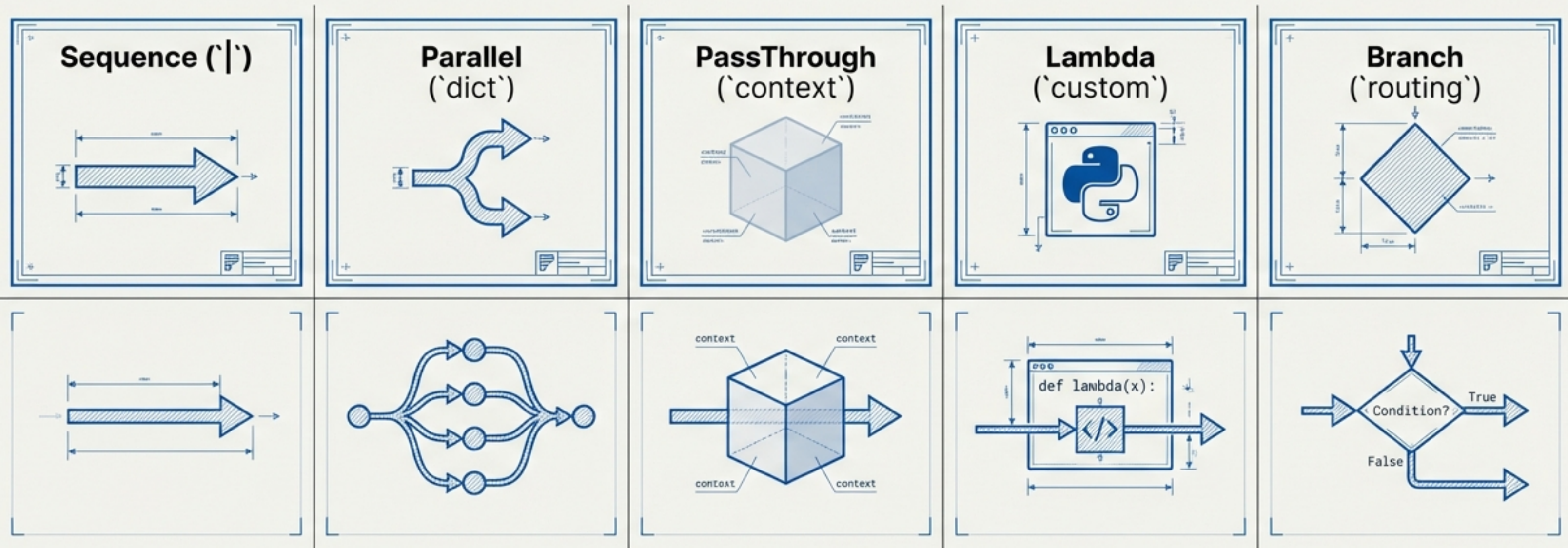
THE LCEL WAY (Declarative)

chain = prompt | model | parser



Unix-style Pipe Syntax for composition.

THE ARCHITECT'S TOOLBOX: SUMMARY



NEXT STEPS:

Using these primitives to build RAG (Retrieval Augmented Generation) Systems.